CSA0503-DATABASE MANAGEMENT SYSTEMS

CO3 AT1

Experiments

1. **For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.**

```
SELECT SID, COUNT(DISTINCT SName)
FROM STUDENT_COURSE
GROUP BY SID
HAVING COUNT(DISTINCT SName) > 1;

SELECT CID, COUNT(DISTINCT CName), COUNT(DISTINCT Instructor)
FROM STUDENT_COURSE
GROUP BY CID
HAVING COUNT(DISTINCT CName) > 1
OR COUNT(DISTINCT Instructor) > 1;
```

```
mysql> SELECT * FROM STUDENT_COURSE;
+-----+---------+------+------------------+------------+-------+
| SID | SName   | CID  | CName            | Instructor | Grade |
+-----+---------+------+------------------+------------+-------+
| 101 | Arun    | C001 | DBMS             | Ravi       | A     |
| 101 | Arun    | C002 | Operating Systems | Kumar      | B     |
| 102 | Divya   | C001 | DBMS             | Ravi       | A+    |
| 102 | Divya   | C003 | Computer Networks | Meena      | A     |
| 103 | Karthik | C002 | Operating Systems | Kumar      | B+    |
+-----+---------+------+------------------+------------+-------+
5 rows in set (0.02 sec)

mysql> SELECT SID, COUNT(DISTINCT SName)
    -> FROM STUDENT_COURSE
    -> GROUP BY SID
    -> HAVING COUNT(DISTINCT SName) > 1;
Empty set (0.07 sec)

mysql> SELECT CID, COUNT(DISTINCT CName), COUNT(DISTINCT Instructor)
    -> FROM STUDENT_COURSE
    -> GROUP BY CID
    -> HAVING COUNT(DISTINCT CName) > 1
    ->     OR COUNT(DISTINCT Instructor) > 1;
Empty set (0.01 sec)
```

2. **Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.**

UNF:

SID | SName | Courses

101 | Arun | C001-DBMS-A, C002-OS-B

1NF:

SID | SName | CID | CName | Grade

101 | Arun | C001 | DBMS | A

101 | Arun | C002 | OS | B

INSERT INTO STUDENT_COURSE_1NF

VALUES

(101, 'Arun', 'C001', 'DBMS', 'A'),

(101, 'Arun', 'C002', 'OS', 'B'),

(102, 'Divya', 'C001', 'DBMS', 'A+'),

(102, 'Divya', 'C003', 'CN', 'A');

```
mysql> CREATE TABLE STUDENT_COURSE_1NF (
    ->     SID INT,
    ->     SName VARCHAR(30),
    ->     CID VARCHAR(10),
    ->     CName VARCHAR(50),
    ->     Grade CHAR(2),
    ->     PRIMARY KEY (SID, CID)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO STUDENT_COURSE_1NF
    -> VALUES
    -> (101, 'Arun', 'C001', 'DBMS', 'A'),
    -> (101, 'Arun', 'C002', 'OS', 'B'),
    -> (102, 'Divya', 'C001', 'DBMS', 'A+'),
    -> (102, 'Divya', 'C003', 'CN', 'A');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_COURSE_1NF;
+-----+--------+------+-------+-------+
| SID | SName  | CID  | CName | Grade |
+-----+--------+------+-------+-------+
| 101 | Arun   | C001 | DBMS  | A     |
| 101 | Arun   | C002 | OS    | B     |
| 102 | Divya  | C001 | DBMS  | A+    |
| 102 | Divya  | C003 | CN    | A     |
+-----+--------+------+-------+-------+
4 rows in set (0.00 sec)
```

3. **Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.**

```
SELECT A, COUNT(DISTINCT B), COUNT(DISTINCT C)
FROM R
GROUP BY A
HAVING COUNT(DISTINCT B) > 1
   OR COUNT(DISTINCT C) > 1;

SELECT B, C, COUNT(DISTINCT D)
FROM R
GROUP BY B, C
HAVING COUNT(DISTINCT D) > 1;

SELECT D, COUNT(DISTINCT E)
FROM R
GROUP BY D
HAVING COUNT(DISTINCT E) > 1;
```

```
mysql> SELECT A, COUNT(DISTINCT B), COUNT(DISTINCT C)
    -> FROM R
    -> GROUP BY A
    -> HAVING COUNT(DISTINCT B) > 1
    ->     OR COUNT(DISTINCT C) > 1;
Empty set (0.00 sec)

mysql> SELECT B, C, COUNT(DISTINCT D)
    -> FROM R
    -> GROUP BY B, C
    -> HAVING COUNT(DISTINCT D) > 1;
Empty set (0.01 sec)

mysql> SELECT D, COUNT(DISTINCT E)
    -> FROM R
    -> GROUP BY D
    -> HAVING COUNT(DISTINCT E) > 1;
Empty set (0.00 sec)

mysql> SELECT * FROM R;
+---+------+------+------+------+
| A | B    | C    | D    | E    |
+---+------+------+------+------+
| 1 |   10 |   20 |   30 |   40 |
| 2 |   11 |   21 |   31 |   41 |
| 3 |   12 |   22 |   32 |   42 |
+---+------+------+------+------+
3 rows in set (0.00 sec)
```

4. **Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.**

```
SELECT
    E.EmpID,
    E.EmpName,
    E.DeptID,
    D.DeptName,
    D.ManagerID
FROM EMPLOYEE E
JOIN DEPARTMENT D
ON E.DeptID = D.DeptID;
```

```
mysql> SELECT * FROM EMPLOYEE;
+-------+---------+--------+
| EmpID | EmpName | DeptID |
+-------+---------+--------+
|     1 | Arun    |     10 |
|     2 | Divya   |     20 |
|     3 | Karthik |     20 |
|     4 | Priya   |     30 |
+-------+---------+--------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM DEPARTMENT;
+--------+----------+-----------+
| DeptID | DeptName | ManagerID |
+--------+----------+-----------+
|     10 | HR       |       101 |
|     20 | IT       |       102 |
|     30 | Finance  |       103 |
+--------+----------+-----------+
3 rows in set (0.00 sec)

mysql> SELECT
    ->     E.EmpID,
    ->     E.EmpName,
    ->     E.DeptID,
    ->     D.DeptName,
    ->     D.ManagerID
    -> FROM EMPLOYEE E
    -> JOIN DEPARTMENT D
    -> ON E.DeptID = D.DeptID;
+-------+---------+--------+----------+-----------+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-------+---------+--------+----------+-----------+
|     1 | Arun    |     10 | HR       |       101 |
|     2 | Divya   |     20 | IT       |       102 |
|     3 | Karthik |     20 | IT       |       102 |
|     4 | Priya   |     30 | Finance  |       103 |
+-------+---------+--------+----------+-----------+
4 rows in set (0.01 sec)
```

5. **Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.**

```
CREATE TABLE R1 (
    C INT PRIMARY KEY,
    D INT
);

CREATE TABLE R2 (
    A INT,
    B INT,
    C INT,
    PRIMARY KEY (A, B),
    FOREIGN KEY (C) REFERENCES R1(C)
);
```

```
mysql> SELECT * FROM R1;
+----+------+
| C  | D    |
+----+------+
| 10 |   20 |
| 11 |   21 |
| 12 |   22 |
+----+------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM R2;
+---+---+------+
| A | B | C    |
+---+---+------+
| 1 | 2 |   10 |
| 3 | 4 |   11 |
| 5 | 6 |   12 |
+---+---+------+
3 rows in set (0.00 sec)
```

6. **Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.**

```
INSERT INTO R1_LOSSLESS
SELECT DISTINCT A, B
FROM R_LOSSLESS;

INSERT INTO R2_LOSSLESS
SELECT DISTINCT A, C
FROM R_LOSSLESS;

SELECT R1.A, R1.B, R2.C
FROM R1_LOSSLESS R1
JOIN R2_LOSSLESS R2
ON R1.A = R2.A;
```

```
mysql> INSERT INTO R1_LOSSLESS
    -> SELECT DISTINCT A, B
    -> FROM R_LOSSLESS;
Query OK, 3 rows affected (0.06 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO R2_LOSSLESS
    -> SELECT DISTINCT A, C
    -> FROM R_LOSSLESS;
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT R1.A, R1.B, R2.C
    -> FROM R1_LOSSLESS R1
    -> JOIN R2_LOSSLESS R2
    -> ON R1.A = R2.A;
+---+------+-----+
| A | B    | C   |
+---+------+-----+
| 1 |   10 | 100 |
| 2 |   20 | 200 |
| 3 |   30 | 300 |
+---+------+-----+
3 rows in set (0.00 sec)
```

7. **Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.**

INSERT INTO TEACHER_SUBJECT_4NF

SELECT DISTINCT TID, Subject

FROM TEACHER_4NF;

INSERT INTO TEACHER_HOBBY_4NF

SELECT DISTINCT TID, Hobby

FROM TEACHER_4NF;

```
mysql> INSERT INTO TEACHER_SUBJECT_4NF
    -> SELECT DISTINCT TID, Subject
    -> FROM TEACHER_4NF;
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO TEACHER_HOBBY_4NF
    -> SELECT DISTINCT TID, Hobby
    -> FROM TEACHER_4NF;
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER_SUBJECT_4NF;
+-----+---------+
| TID | Subject |
+-----+---------+
| T01 | DBMS    |
| T01 | OS      |
| T02 | AI      |
| T02 | ML      |
+-----+---------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM TEACHER_HOBBY_4NF;
+-----+---------+
| TID | Hobby   |
+-----+---------+
| T01 | Cricket |
| T01 | Music   |
| T02 | Reading |
+-----+---------+
3 rows in set (0.00 sec)
```

8. **Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).**

```
INSERT INTO SP_5NF
SELECT DISTINCT Supplier, Part
FROM SUPPLY_5NF;


SELECT
   SP.Supplier,
   SP.Part,
   SJ.Project
FROM SP_5NF SP
JOIN SJ_5NF SJ
   ON SP.Supplier = SJ.Supplier
JOIN PJ_5NF PJ
   ON SP.Part = PJ.Part
   AND SJ.Project = PJ.Project;
```

```
mysql> SELECT
    ->     SP.Supplier,
    ->     SP.Part,
    ->     SJ.Project
    -> FROM SP_5NF SP
    -> JOIN SJ_5NF SJ
    ->     ON SP.Supplier = SJ.Supplier
    -> JOIN PJ_5NF PJ
    ->     ON SP.Part = PJ.Part
    ->     AND SJ.Project = PJ.Project;
+----------+------+---------+
| Supplier | Part | Project |
+----------+------+---------+
| S1       | P1   | J1      |
| S1       | P1   | J2      |
| S1       | P2   | J1      |
| S1       | P2   | J2      |
+----------+------+---------+
4 rows in set (0.01 sec)
```

9. **Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.**

```
SELECT
    S.StudentID,
    S.StudentName,
    C.CourseID,
    C.CourseName,
    I.InstructorID,
    I.InstructorName
FROM COLLEGE_STUDENT_3NF S
JOIN COLLEGE_ENROLLMENT_3NF E
    ON S.StudentID = E.StudentID
JOIN COLLEGE_COURSE_3NF C
    ON E.CourseID = C.CourseID
JOIN COLLEGE_INSTRUCTOR_3NF I
    ON C.InstructorID = I.InstructorID;
```

```
mysql> SELECT
    ->     S.StudentID,
    ->     S.StudentName,
    ->     C.CourseID,
    ->     C.CourseName,
    ->     I.InstructorID,
    ->     I.InstructorName
    -> FROM COLLEGE_STUDENT_3NF S
    -> JOIN COLLEGE_ENROLLMENT_3NF E
    ->     ON S.StudentID = E.StudentID
    -> JOIN COLLEGE_COURSE_3NF C
    ->     ON E.CourseID = C.CourseID
    -> JOIN COLLEGE_INSTRUCTOR_3NF I
    ->     ON C.InstructorID = I.InstructorID;
+-----------+-------------+----------+------------+--------------+----------------+
| StudentID | StudentName | CourseID | CourseName | InstructorID | InstructorName |
+-----------+-------------+----------+------------+--------------+----------------+
|       101 | Arun        | C001     | DBMS       | I01          | Ravi           |
|       102 | Divya       | C001     | DBMS       | I01          | Ravi           |
|       103 | Karthik     | C002     | OS         | I02          | Kumar          |
+-----------+-------------+----------+------------+--------------+----------------+
3 rows in set (0.01 sec)
```

10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

```
CREATE TABLE FD_CLOSURE (
    Determinant VARCHAR(20),
    Dependent VARCHAR(20)

INSERT INTO FD_CLOSURE VALUES
('A', 'B'),
('A', 'C'),
('BC', 'D'),
('D', 'E');

SELECT * FROM FD_CLOSURE;
```

```
mysql> CREATE TABLE FD_CLOSURE (
    ->     Determinant VARCHAR(20),
    ->     Dependent VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO FD_CLOSURE VALUES
    -> ('A', 'B'),
    -> ('A', 'C'),
    -> ('BC', 'D'),
    -> ('D', 'E');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM FD_CLOSURE;
+-------------+-----------+
| Determinant | Dependent |
+-------------+-----------+
| A           | B         |
| A           | C         |
| BC          | D         |
| D           | E         |
+-------------+-----------+
4 rows in set (0.00 sec)
```

);